

Сдача проектной работы третьего спринта

Создайте бэкенд приложения-блога с использованием Spring Framework и добавьте тесты.

Мы подготовили для вас архив с фронтендом приложения-блога, написанном с React.

Структура архива:

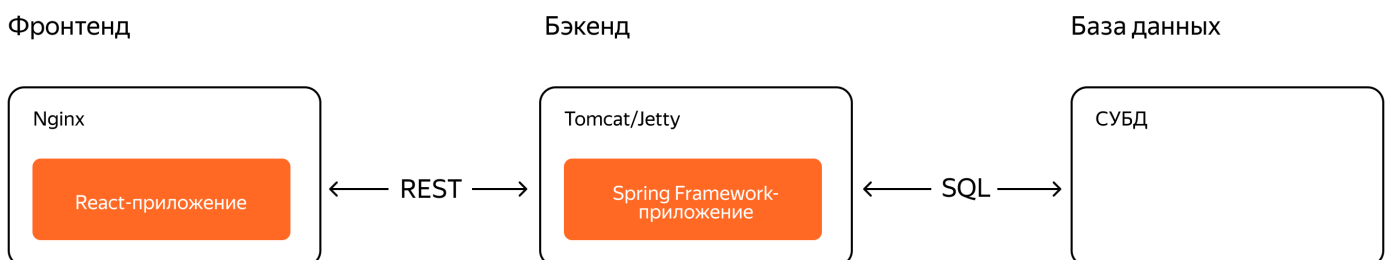
`dist` — директория со статическим контентом фронтенда (`index.html`, JavaScript и CSS файлы);

`nginx.conf` — конфиг Nginx с настройками приложения (ссылается на `index.html`);

`Dockerfile` содержит инструкции для создания образа Docker с фронтендом на базе веб-сервера Nginx;

`docker-compose.yaml` — конфигурационный файл для запуска контейнера с фронтендом, используемый инструментом Docker Compose.

Архитектура приложения-блога:



Фронтенд работает на веб-сервере Nginx, принимает запросы от браузера пользователя на <http://localhost:80/>, отправляет запросы на получение данных в бэкенд по REST.

Бэкенд работает в сервлет-контейнере Tomcat/Jetty, принимает REST-запросы от фронтенда на <http://localhost:8080/>, отправляет SQL-запросы на создание/получение/изменение/удаление данных в БД PostgreSQL/H2 или любой другой, формирует ответ и возвращает фронтенду.

База данных хранит данные, обрабатывает запросы от бэкенда и возвращает результат.

Напишите бэкенд приложения-блога на Java с использованием Spring Framework, работающий в сервлет-контейнере Tomcat/Jetty, выберите БД для хранения постов и комментариев и реализуйте интеграцию бэкенда с ней.

Как запускать фронтенд:

Распаковать zip-архив.

Перейти в директорию с `docker-compose.yaml`.

Запустить в терминале `docker compose up -d`.

Запустить в терминале `docker ps` и убедиться, что контейнер с фронтендом запущен. Вы должны увидеть вывод, аналогичный представленному ниже:

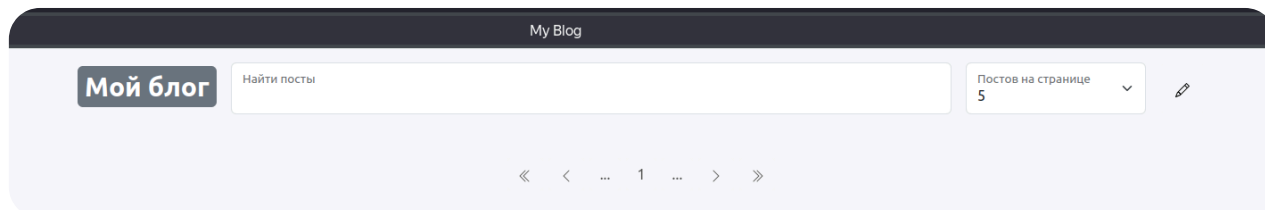
```
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
f09570d2cc25   docker-my-blog-front-app            "/docker-entrypoint...." 5 hours
```

Фронтенд запущен!

Остановить фронтенд можно командой `docker compose down` в терминале.

Далее реализуйте бэкенд. После реализации бэкенда вы сможете использовать фронтенд:

Откройте браузер и зайдите на <http://localhost>:



Нажмите на **карандаш** — откроется форма добавления поста:

картинка — выберите картинку поста;

название — укажите название поста;

текст — напишите текст поста в формате Markdown;

теги поста — введите теги поста (разделяются через пробелы и переносы строки).

Добавление поста

Картинка

Choose File konzhak.jpg

Название

Конжакомкий камень

Текст

Материал из Википедии — свободной энциклопедии

****Конжаковский Камень**** — гора в южной части Северного Урала, в городском округе Карпинск Свердловской области (Россия). Одна из высочайших вершин Уральских гор (1569 м). Сложена пироксенитами, дунитами и габбро. В нижней части склоны покрыты хвойными лесами, а с высоты 900—1000 м — горной тундрой и каменными россыпями. Названа по имени охотника-вогула Конжакова, юрта которого некогда стояла у основания горы.

Превью

Материал из Википедии — свободной энциклопедии

Конжаковский Камень — гора в южной части Северного Урала, в городском округе Карпинск Свердловской области (Россия). Одна из высочайших вершин Уральских гор (1569 м). Сложена пироксенитами, дунитами и габбро. В нижней части склоны покрыты хвойными лесами, а с высоты 900—1000 м — горной тундрой и каменными россыпями. Названа по имени охотника-вогула Конжакова, юрта которого некогда стояла у основания горы.

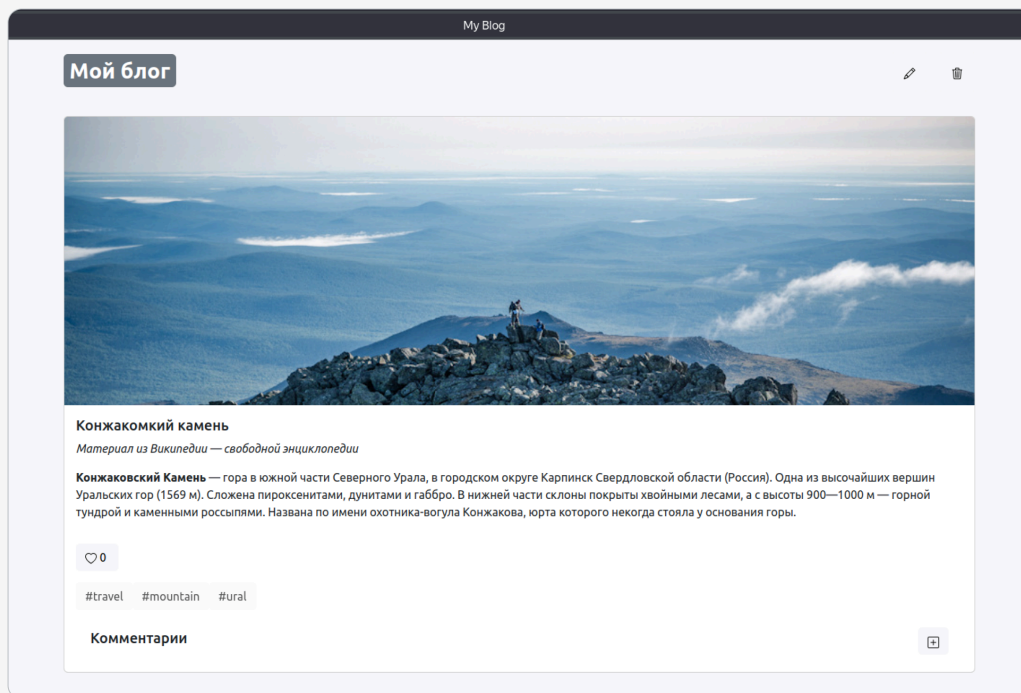
Теги

travel mountain ural

#travel #mountain #ural

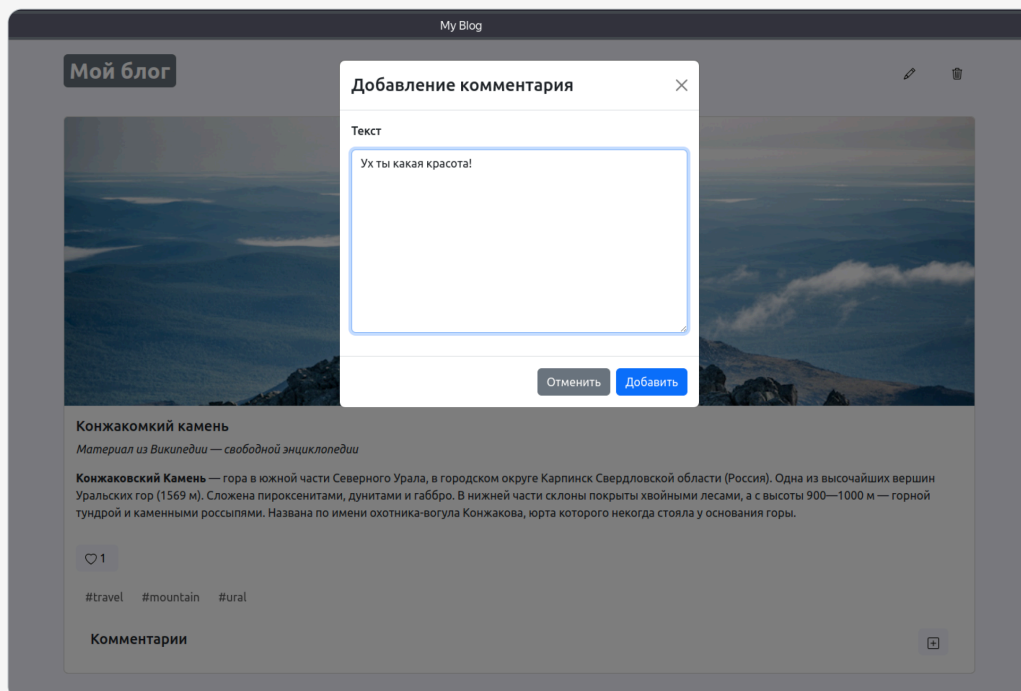
Отменить Добавить

Нажмите **Добавить** — создастся пост и автоматически откроется страница поста:

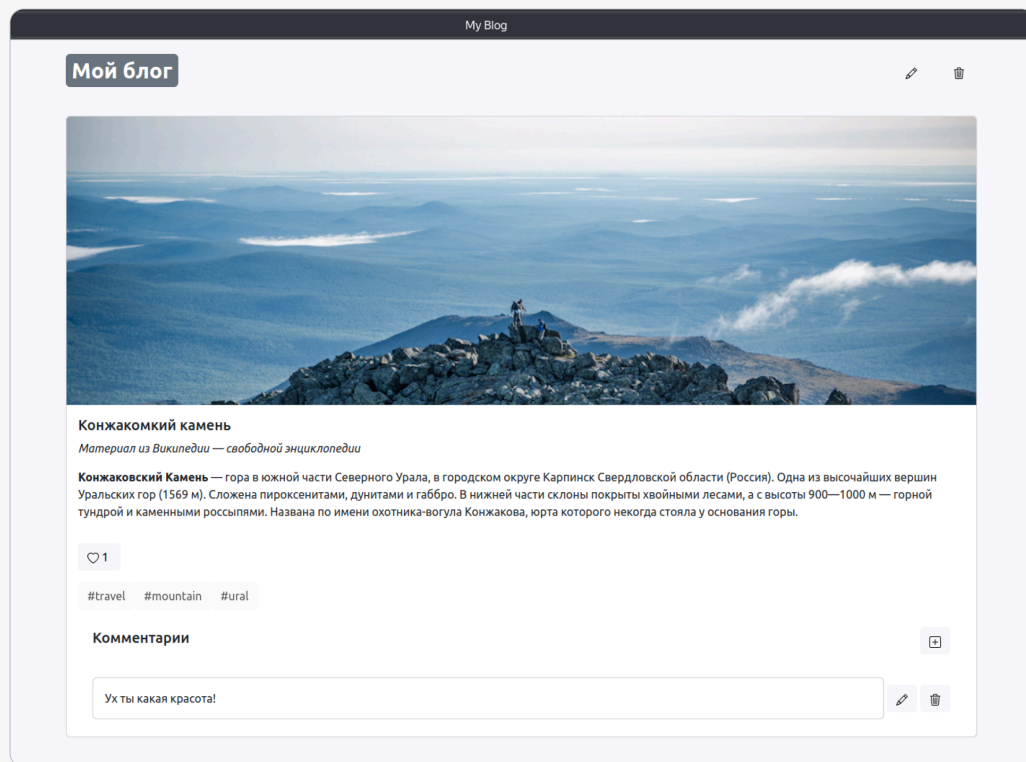


Нажмите на сердечко — добавится лайк посту (можно нажимать неограниченное количество раз).

Нажмите на + — откроется форма добавления комментария, в которой можно ввести текст комментария:



Нажмите **Добавить** — создастся комментарий и появится в списке комментариев:



Комментарий можно отредактировать, нажав на **карандаш** напротив — откроется форма редактирования комментария (аналогичная форме добавления).

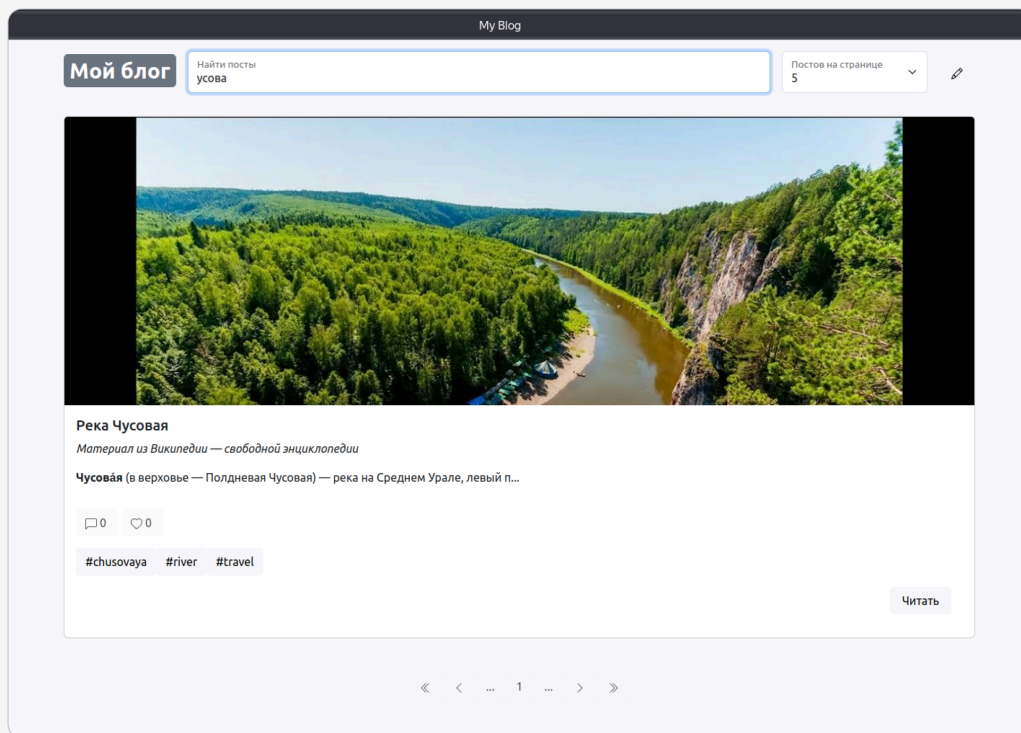
Комментарий можно удалить, нажав на **корзину** напротив — комментарий удалится из списка комментариев.

Пост можно отредактировать, нажав на **карандаш** в правом верхнем углу экрана — откроется форма редактирования поста (аналогичная форме добавления).

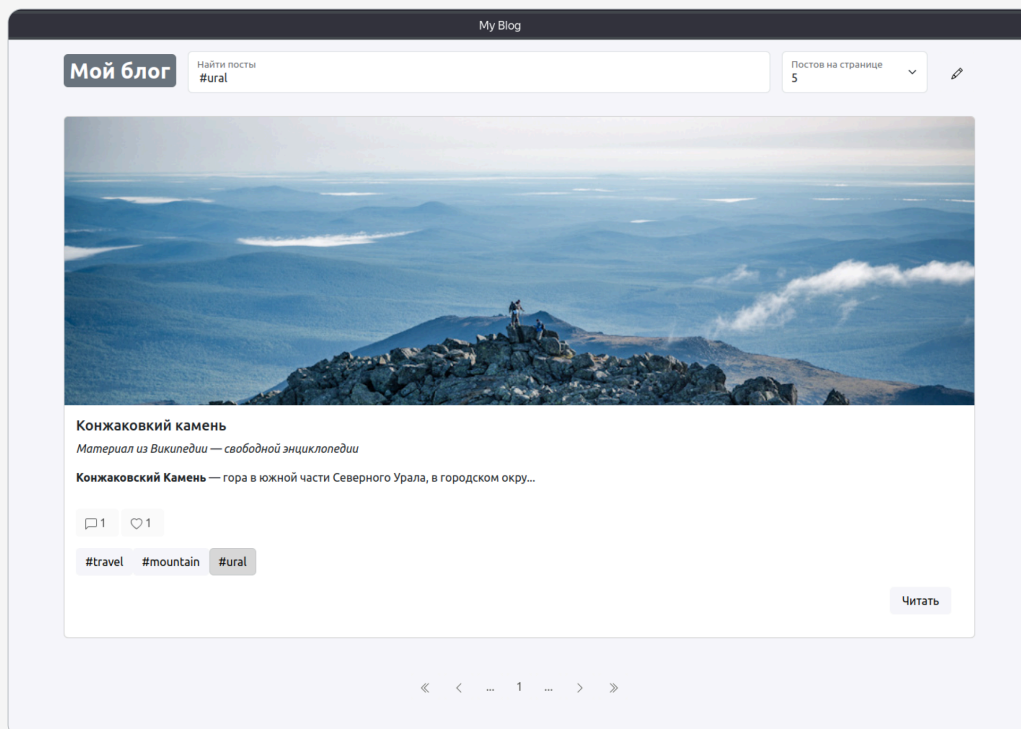
Пост можно удалить, нажав на **корзину** в правом верхнем углу экрана — пост удалится, произойдёт автоматический переход на главную страницу (ленту постов).

Можно нажать в браузере **назад** и вернуться в ленту постов.

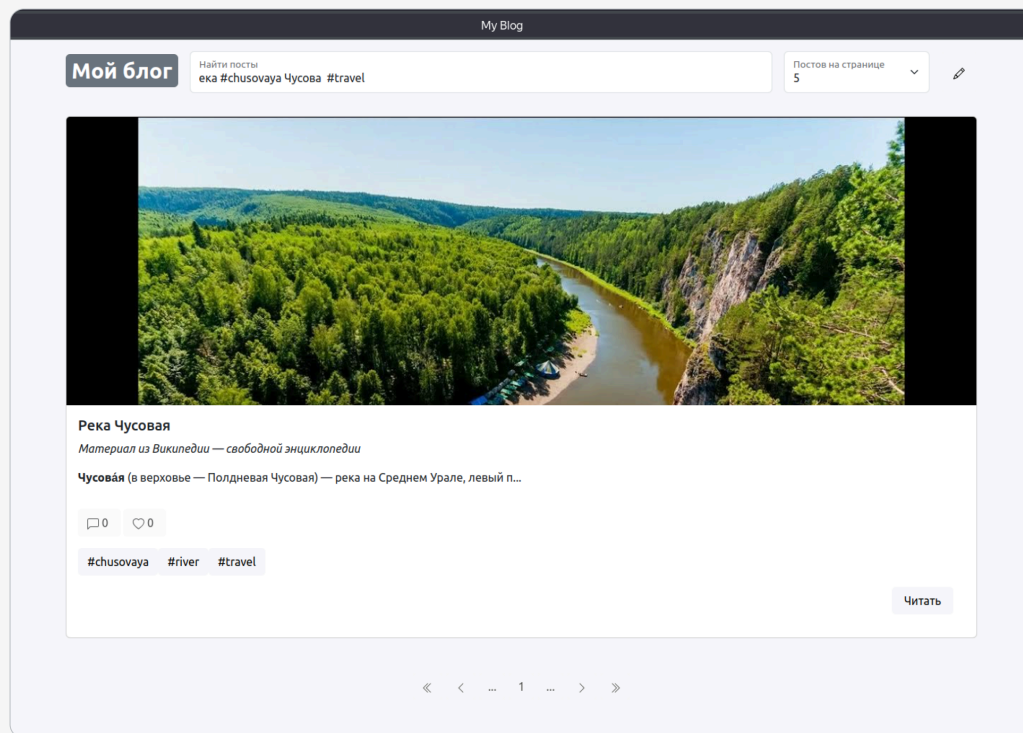
Если начать вводить название в строку поиска, то произойдёт автоматическая фильтрация постов по вхождению введённой подстроки в название поста:



Если нажать на тег, то он автоматически подставится в поле поиска, и произойдёт автоматическая фильтрация постов по наличию тега в посте:



Возможен поиск и по вхождению подстроки в название, и по наличию тегов одновременно:



Правила поиска такие:

Строка поиска разбивается на слова по пробелам.

Пустые слова удаляются из поиска.

Слова, начинающиеся с #, считаются тегами, и посты фильтруются по ним по «И».

Слова, не начинающиеся с #, склеиваются вместе через пробел и считаются подстрокой поиска по названию.

Фильтрация постов по подстроке и тегам происходит по «И».

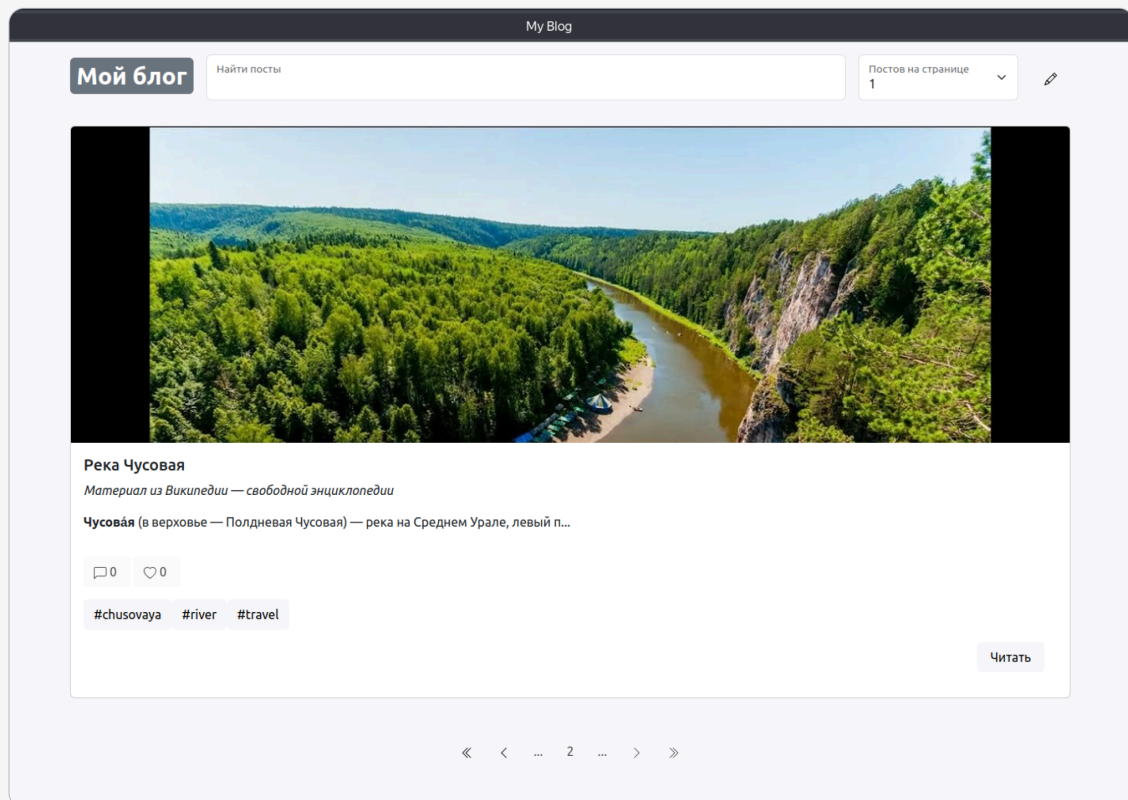
15. Если число постов на странице превышает выбранное значение (селект в правом верхнем углу), то становятся доступными кнопки внизу страницы перехода:

<< — на первую страницу;

< — на предыдущую страницу;

> — на следующую страницу;

>> — на последнюю страницу.



Требования к бэкенду и его функциональность

Бэкенд веб-приложения блога написан с использованием Spring Framework (версия 6.1 и выше), выполняющегося в любом из современных сервлет-контейнеров Tomcat/Jetty (на ваш выбор). **Не используйте в этом проекте Spring Boot — он понадобится в следующем спринте.**

Код приложения хранится в Git-репозитории (на Github).

Сборка приложения осуществляется с помощью систем сборки Maven/Gradle (на ваш выбор).

Код приложения написан на Java 21.

База данных приложения может быть как персистентной (например, PostgreSQL), так и в памяти (например, H2).

Бэкенд получает запросы от фронтенда по REST, обрабатывает их, формирует ответ и возвращает его фронтенду.

Бэкенд предоставляет следующие эндпоинты, которые использует фронтенд:

Получение списка постов (на главной странице — ленте постов) — метод `GET` : `/api/posts?search=Lalala&pageNumber=1&pageSize=5`, где

`search` — это строка из поля поиска, `pageNumber` — номер текущей страницы, `pageSize` — число постов на странице (все поля **обязательные**). Бэкенд отдаёт ответ в формате JSON:

```
{
  "posts": [
    {
      "id": 1,
      "title": "Название поста 1",
      "text": "Текст поста в формате Markdown...",
      "tags": ["tag_1", "tag_2"],
      "likesCount": 5,
      "commentsCount": 1
    },
    {
      "id": 2,
      "title": "Название поста 2",
      "text": "Текст поста в формате Markdown...",
      "tags": [],
      "likesCount": 1,
      "commentsCount": 5
    }
  ],
  "hasPrev": true,
  "hasNext": false,
  "lastPage": 3
}
```

Здесь `posts` — список постов, `hasPrev` — true, если текущая страница **не первая**, `hasNext` — true, если текущая страница **не последняя**, `lastPage` — номер последней страницы, `id` — идентификатор поста, `title` — название поста, `text` — текст поста (в формате Markdown, если больше 128 символов, то обрезается до 128 символов и добавляется «...»), `tags` — список тегов, `likesCount` — число лайков поста, `commentsCount` — число комментариев поста (все поля **обязательные**).

Получение поста (на странице поста) — метод `POST`: `/api/posts/1`, где `1` — идентификатор поста. Бэкенд отдаёт ответ в формате JSON:

```
{
  "id": 1,
  "title": "Название поста 1",
  "text": "Текст поста в формате Markdown...",
  "tags": \["tag_1", "tag_2"\],
  "likesCount": 5,
  "commentsCount": 1
}
```

Здесь `id` — идентификатор поста, `title` — название поста, `text` — текст поста (в формате Markdown, если больше 128 символов, текст **не обрезаётся**), `tags` — список тегов, `likesCount` — число лайков поста, `commentsCount` — число комментариев поста (все поля **обязательные**).

Добавление поста (на главной странице — ленте постов) — метод `POST`: `/api/posts`. Фронт присылает запрос в формате JSON:

```
{
  "title": "Название поста 3",
  "text": "Текст поста в формате Markdown...",
  "tags": ["tag_1", "tag_2"]
}
```

Здесь `title` — название поста, `text` — текст поста (в формате Markdown), `tags` — список тегов (все поля **обязательные**). Бэкенд создаёт пост и отдаёт ответ в формате JSON:

```
{
  "id": 3,
  "title": "Название поста 3",
  "text": "Текст поста в формате Markdown...",
  "tags": ["tag_1", "tag_2"],
  "likesCount": 0,
  "commentsCount": 0
}
```

`id` — сгенерированный идентификатор поста, `title` — название поста, `text` — текст поста (в формате Markdown), `tags` — список тегов,

`likesCount` — всегда 0, `commentsCount` — всегда 0 (все поля обязательные).

Редактирование поста (на странице поста) — метод `PUT`:

`/api/posts/3`. Фронт присылает запрос в формате JSON:

```
{
  "id": 3,
  "title": "Название поста 3",
  "text": "Текст поста в формате Markdown...",
  "tags": ["tag_1", "tag_2"]
}
```

`id` — идентификатор поста, `title` — название поста, `text` — текст поста (в формате Markdown), `tags` — список тегов (все поля **обязательные**). Бэкенд редактирует текст поста и отдаёт ответ в формате JSON:

```
{
  "id": 3,
  "title": "Название поста 3",
  "text": "Текст поста в формате Markdown...",
  "tags": ["tag_1", "tag_2"],
  "likesCount": 0,
  "commentsCount": 0
}
```

`id` — идентификатор поста, `title` — название поста, `text` — текст поста (в формате Markdown), `tags` — список тегов, `likesCount` — число лайков поста, `commentsCount` — число комментариев поста (все поля **обязательные**).

Удаление поста (на странице поста) — метод `DELETE`:

`/api/posts/1`, где `1` — идентификатор поста. Бэкенд должен удалить пост со всеми комментариями и вернуть `200 OK`.

Инкремент числа лайков поста (на странице поста) — метод `POST`:

`/api/posts/1/likes`, где `1` — идентификатор поста. Бэкенд должен добавить +1 к числу лайков поста и вернуть обновлённое число лайков поста (число в теле ответа).

Обновление картинки поста (при добавлении и редактировании поста) — метод `PUT`: `/api/posts/1/image`, где `1` — идентификатор поста. Фронтенд присылает в теле запроса файл в формате `multipart/form-data`, где `Content-Disposition: form-data; name="image"; filename="image_name.jpg"`. Бэкенд должен обновить картинку поста и вернуть `200 Ok`.

Получение картинки поста (главная страница — лента постов и страница поста) — метод `GET`: `/api/posts/1/image`, где `1` — идентификатор поста. Бэкенд должен вернуть массив байт картинки поста в теле ответа.

Получение комментариев поста (на странице поста) — метод `GET`: `/api/posts/1/comments`, где `1` — идентификатор поста. Бэкенд должен отдать ответ в формате JSON:

```
[
  {
    "id": 1,
    "text": "Комментарий к посту 1",
    "postId": 1
  },
  {
    "id": 2,
    "text": "Ещё один комментарий к посту 1",
    "postId": 1
  }
]
```

`id` — идентификатор комментария, `text` — текст комментария, `postId` — идентификатор поста (все поля **обязательные**).

Получение комментария поста (на странице поста) — метод `GET`: `/api/posts/1/comments/2`, где `1` — идентификатор поста, `2` — идентификатор комментария. Бэкенд отдаёт ответ в формате JSON:

```
{
  "id": 2,
  "text": "Ещё один комментарий к посту 1",
  "postId": 1
}
```


`id` — идентификатор комментария, `text` — текст комментария, `postId` — идентификатор поста (все поля **обязательные**).

Добавление комментария к посту (на странице поста) — метод

`POST`: `/api/posts/1/comments`, где `1` — идентификатор поста.

Фронт присылает запрос в формате JSON:

```
{
  "text": "Комментарий к посту",
  "postId": 1
}
```

`text` — текст комментария, `postId` — идентификатор поста (все поля **обязательные**). Бэкенд создаёт новый комментарий к посту и возвращает ответ в формате JSON:

```
{
  "id": 2,
  "text": "Ещё один комментарий к посту 1",
  "postId": 1
}
```

`id` — сгенерированный идентификатор комментария, `text` — текст комментария, `postId` — идентификатор поста (все поля **обязательные**).

Редактирование комментария к посту (страница поста) — метод

`PUT`: `/api/posts/1/comments/2`, где `1` — идентификатор поста, `2`

— идентификатор комментария. Фронтенд передаёт запрос в формате JSON:

```
{
  "id": 2,
  "text": "Второй комментарий к посту 1",
  "postId": 1
}
```

`id` — идентификатор комментария, `text` — текст комментария, `postId` — идентификатор поста (все поля **обязательные**). Бэкенд редактирует комментарий к посту и возвращает ответ в формате JSON:

```
{
  "id": 2,
  "text": "Второй комментарий к посту 1",
  "postId": 1
}
```

`id` — идентификатор комментария, `text` — текст комментария, `postId` — идентификатор поста (все поля **обязательные**).

Удаление комментария (страница поста) — метод `DELETE`:
`/api/posts/1/comments/2`, где `1` — идентификатор поста, `2` — идентификатор комментария. Бэкенд удаляет комментарий и возвращает `200 OK`.

Бэкенд запущен на <http://localhost:8080>.

Бэкенд покрыт тестами (юнит- и интеграционными) с использованием JUnit 5, TestContext Framework и кеширования контекстов.

Как работать над заданием

Изучить описание задания по написанию бэкенда приложения-блога (REST-эндпоинты, функциональность).

По изученному заданию спроектировать и реализовать бэкенд в соответствии с требованиями проекта.

Стараться использовать микрокоммиты при разработке приложения.

После финального коммита сделать пуш на GitHub.

Что нужно сделать

Создать новый репозиторий в своём профиле на GitHub для проекта (например, с названием `my-blog-back-app`). Профиль должен быть **публичным**, чтобы ревьюер мог с ним работать.

Клонировать репозиторий себе в локальный Git-репозиторий. Вся работа над проектом должна вестись в отдельной ветке в соответствии с парадигмой GitFlow.

Создать Maven/Gradle-проект в локальном репозитории.

Добавить необходимые зависимости на Spring Framework, Spring Test Framework, Spring Web, Spring Data JDBC, JUnit 5. Добавить свои другие зависимости, которые помогут при написании кода и разработке приложения (например, JDBC-драйвер для вашей БД, Lombok).

Открыть проект в среде разработки, например, в IntelliJ Idea.

Добавить в файл `.gitignore` директории/файлы, которые не должны быть в Git-репозитории.

Добавить файл `README.md` с кратким описанием проекта.

Сделать init-коммит в Git-репозиторий и отправить изменения на GitHub.

Спроектировать слои приложения согласно паттернам разработки корпоративных приложений (Controller, Service, Model, DAO).

Спроектировать классы/подмодули в каждом слое согласно принципам и паттернам разработки SOLID, YAGNI и другим.

Спроектировать структуру БД для хранения данных блога: посты, комментарии.

Добавить инфраструктуру для создания `war`-файла для запуска в сервлет-контейнере (сборка `war` в Maven/Gradle, `web.xml` и так далее).

Добавить классы Java-конфигурации для интеграции Spring-приложения с выбранным сервлет-контейнером (написать соответствующие инфраструктурные интеграционные бины/классы, прописать классы в `web.xml`, которые будут поднимать контекст).

Добавить классы Java-конфигурации для интеграции Spring-приложения с выбранной СУБД (написать соответствующие инфраструктурные интеграционные бины/классы, создать ресурсы, SQL-скрипты для создания структуры БД при старте приложения, передачу пароля для доступа к БД).

Написать классы слоя модели, DAO, сервисов, контроллеров, добавить их в контекст приложения.

Написать тесты на слой сервисов и его взаимодействие со слоем модели с использованием Spring Test Framework.

Написать базовые интеграционные тесты (на MVC и DAO) на типовые сценарии (добавление поста, удаление, фильтрация постов и т. д.) с использованием WebMvc и Embedded In-Memory базы данных H2.

Написать сборку `war` и его деплой в сервлет-контейнер.

Дописать `README.md`: как собирать бэкенд, запускать тесты, деплоить бэкенд в сервлет-контейнер, запускать и использовать бэкенд.

В процессе разработки бэкенда делать микрокоммиты в Git, дописывать файлы описания сборки Maven/Gradle, накатки схемы БД и так далее.

Что проверит ревьюер

Правильность описания проекта в `pom.xml` и `build.gradle`.

Структуру проекта согласно слоям приложения и правильное расположение и взаимодействие между слоями.

Структуру и взаимодействие классов согласно принципам и паттернам разработки корпоративных приложений.

Наименование классов, методов, полей и других элементов в соответствии с их назначением.

Правильность использования Spring Framework для разработки web-приложения.

Правильность проектирования структуры БД для хранения постов и комментариев к ним.

Правильность интеграции бэкенда с фронтендом по REST.

Покрытие тестами слоёв сервисов и модели с использованием Spring Test Framework.

Правильность написания базовых тестов на MVC и DAO с использованием Spring Test Framework.

Максимальное использование кеширования контекстов при прогоне тестов.

Подробность и понятность описания проекта в `README.md`, форматирование.

Использование микрокоммитов при разработке приложения.

Правильное использование rebase/merge-слияний в истории проекта.

Тесты приложения при прогоне «зелёные».

Как сдавать проект

После завершения работы ваш репозиторий с проектом должен находиться в вашем аккаунте на GitHub и быть **публичным**.

Работа над проектом ведётся в соответствии с парадигмой GitFlow:

У вас есть основная ветка `main`, в которой находятся итоговые изменения по вашему проекту.

В начале работы над проектом третьего спринта создайте новую ветку от `main` (например, с названием `module_one_sprint_three_branch`).

Далее вся работа над проектом ведётся в этой ветке — в ней должна отображаться вся история коммитов.

После завершения работы над проектом сделайте Pull request о мёрдже этой ветки (`module_one_sprint_three_branch`) в `main`.

Ссылку на Pull request нужно прислать ревьюеру в тренажёре этого урока-проекта.

Далее все изменения, связанные с замечаниями ревьюера, оформляются в виде коммитов в ветку проекта (`module_one_sprint_three_branch`).

После того, как ревьюер принял ваш проект, нужно смёрджить Pull request в ветку `main`.

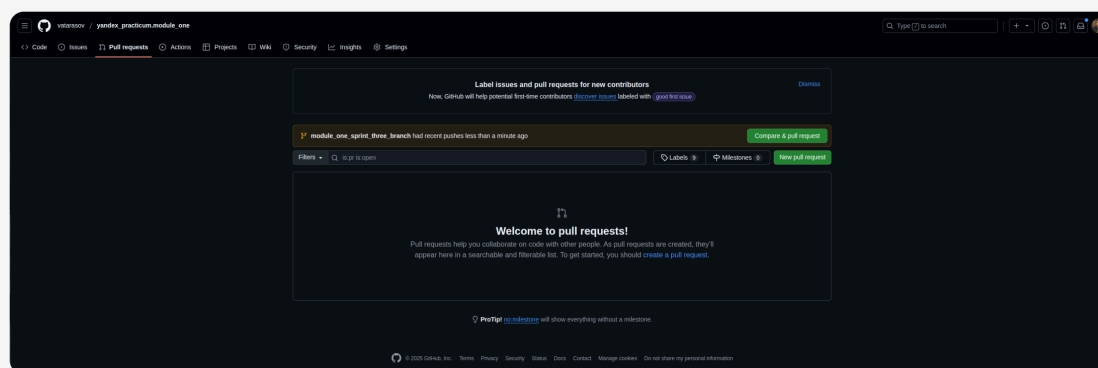
При работе над проектом четвёртого спринта нужно будет сделать то же самое, но в ветку `main` к этому моменту будут смёрджены результаты работы по третьему спринту.

Например, путь до вашего репозитория:

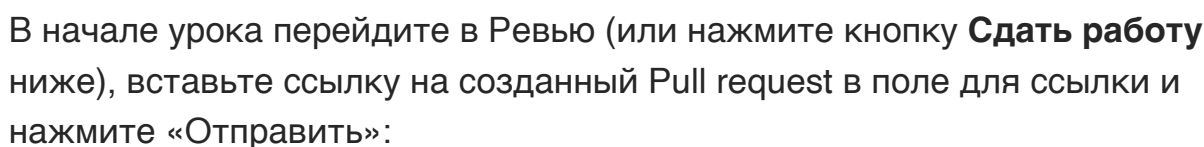
https://github.com/vatarasov/yandex_practicum.module_one.

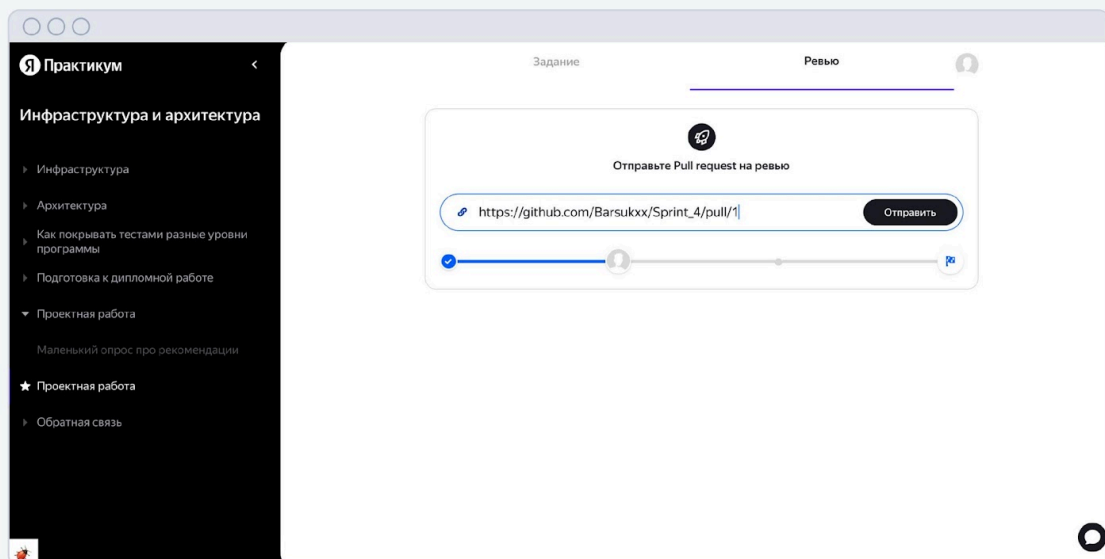
После завершения работы над проектом создайте Pull request вашей ветки в ветку `main`:

Нажмите **New pull request**:



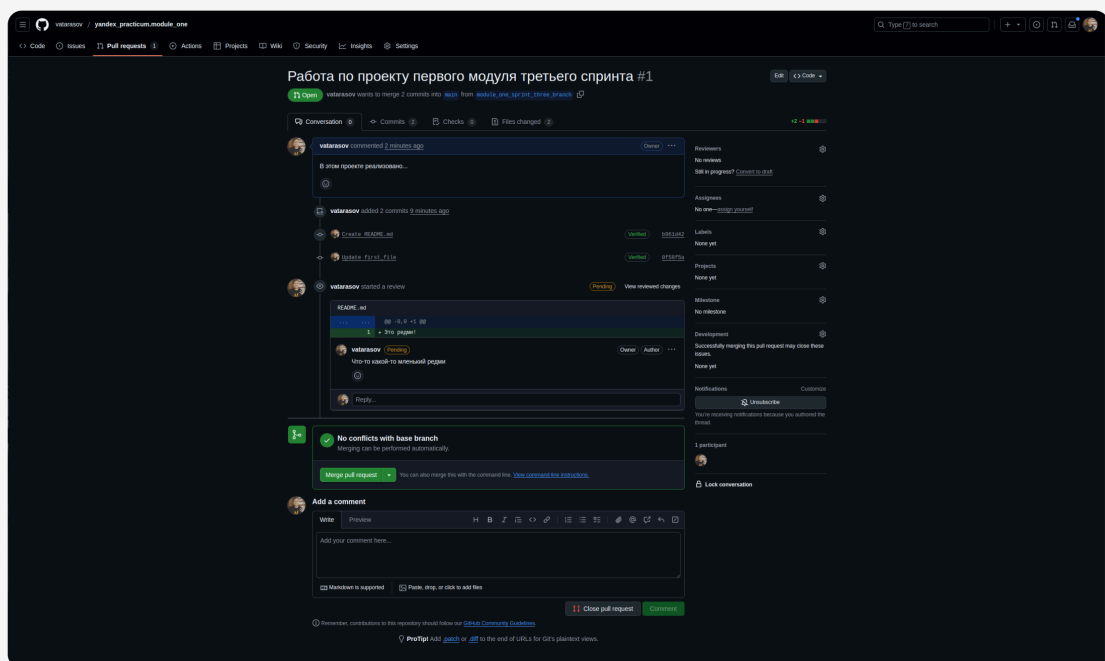
Выберите ветку с проектом (`module_one_sprint_three_branch`) и нажмите **Create pull request**:



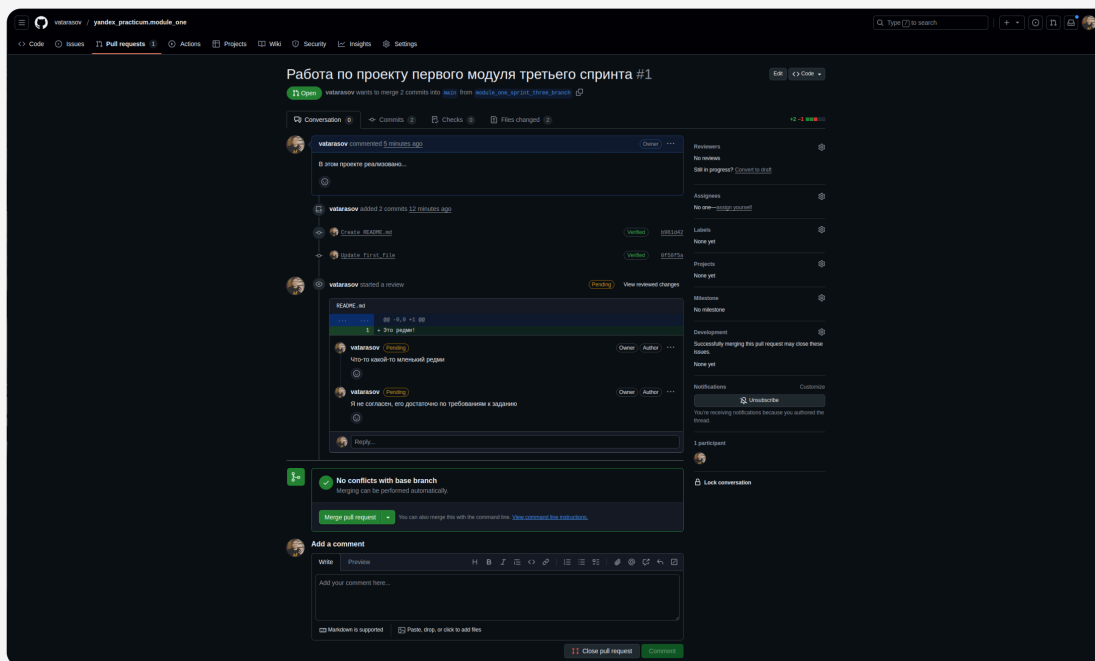


Дождитесь проверки вашего проекта ревьюером.

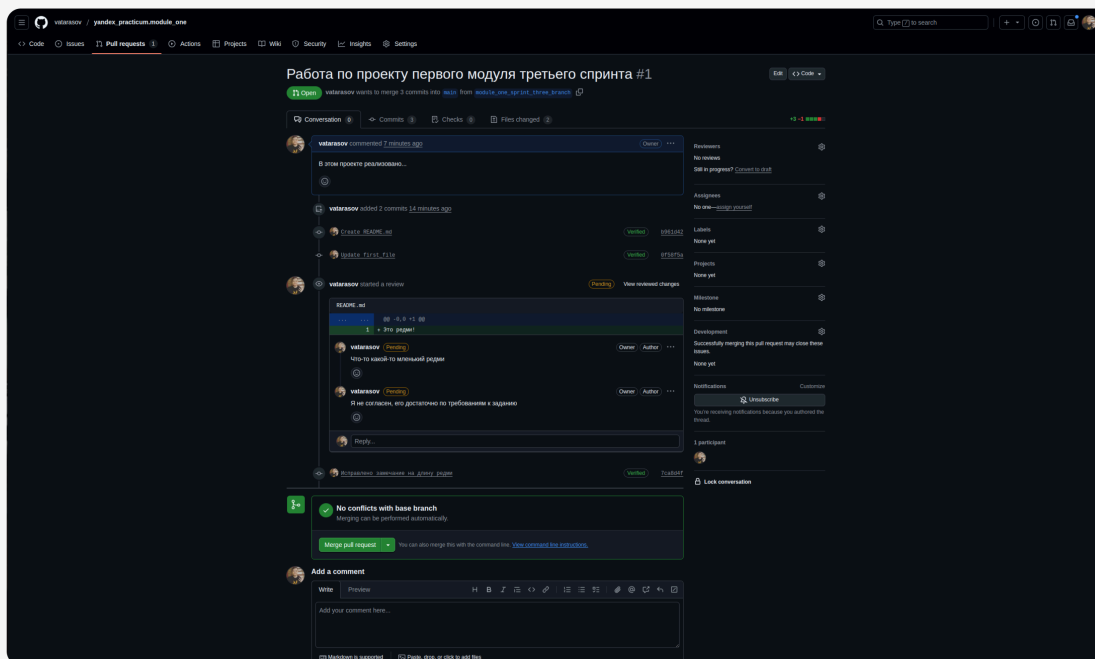
Если по проекту есть замечания, то подробные комментарии будут оформлены в пулл-реквесте (в тренажёре будет только общий комментарий):



Вся работа по замечаниям ведётся в пулл-реквесте (ответы на комментарии, обсуждения изменений и т. д.):

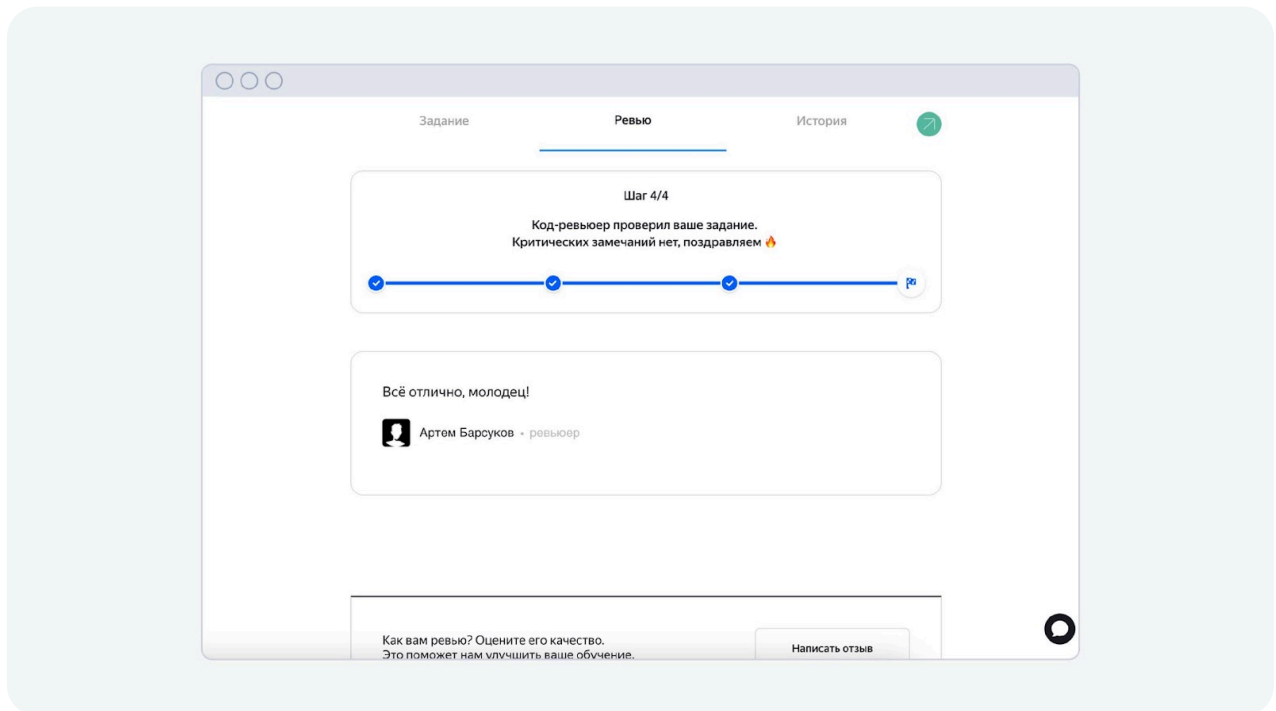


Чтобы исправить замечания, делайте коммиты в эту же ветку с пояснениями о правках:



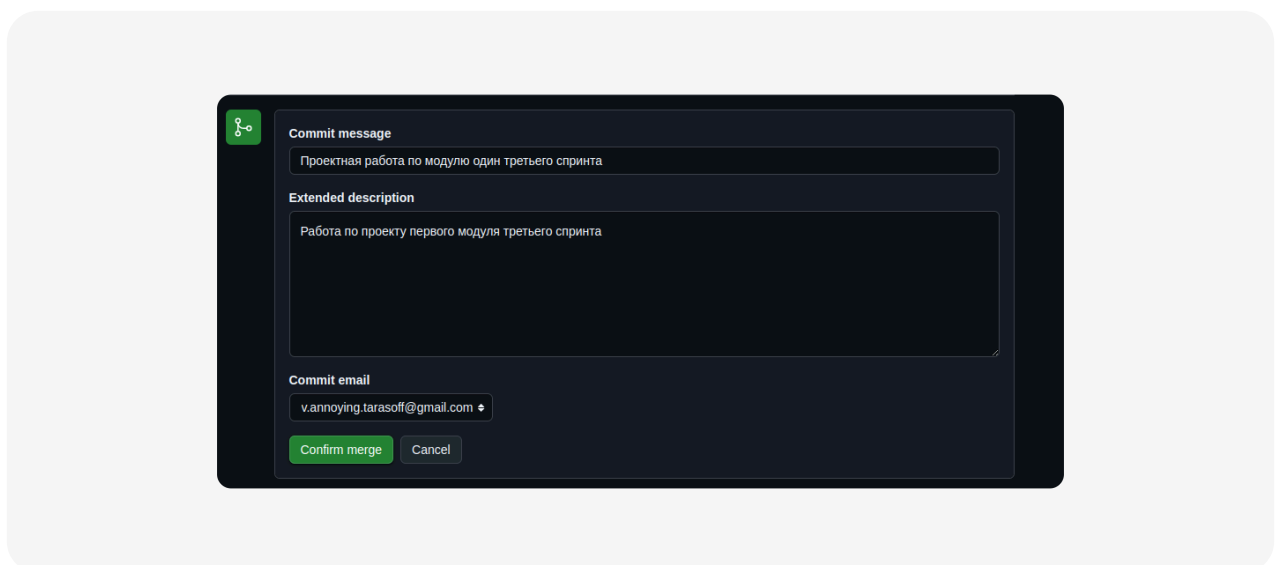
Ревьюер проверит правки, и если замечания останутся, то напишет комментарии о том, что ещё необходимо сделать.

После исправления всех замечаний вы увидите такое сообщение — проектная работа сдана:

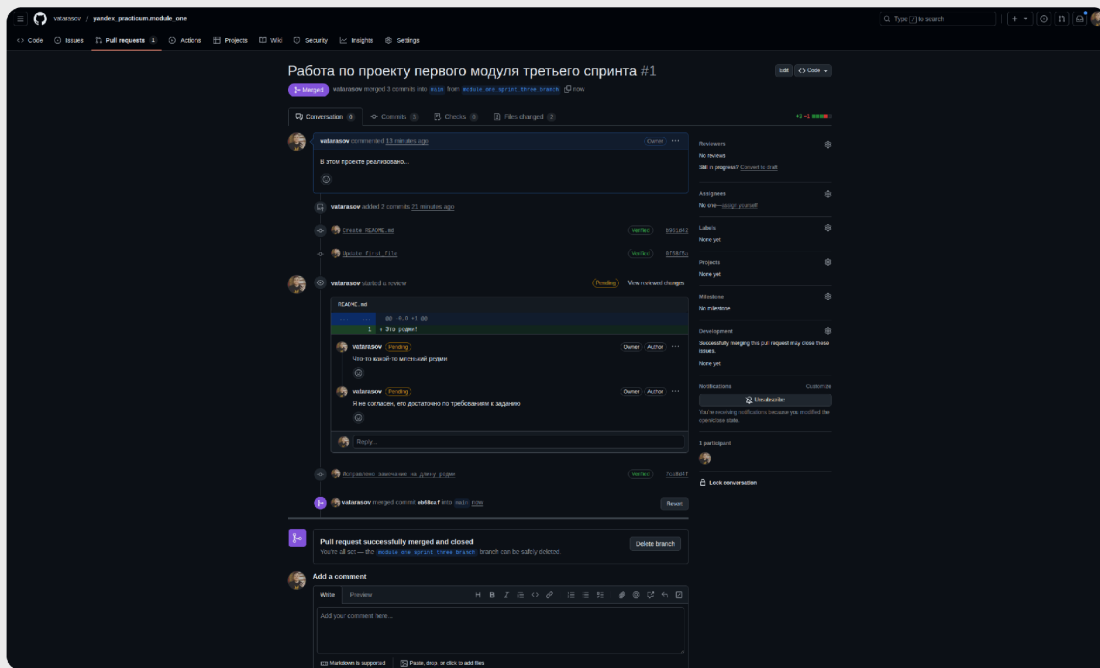


Далее смёрджите ваш пулл-реквест в main.

Нажмите кнопку **Merge pull request**, заполните название коммита и описание:



Нажмите кнопку **Confirm merge**:



Успех!